

Greedy Templates

Two interval templates plus non-interval greedy patterns.

Quick Reference Table

#	Name	Sort by	Greedy rule	Key condition	Return
43 5	Non-overlapping Intervals	end	keep earliest-ending non-overlapping	<code>start >= lastEnd</code> → keep	int (removed count)
45 2	Min Arrows to Burst Balloons	end	one arrow covers all overlapping	<code>start > arrowEnd</code> → new arrow	int (arrows)
64 6	Max Length Pair Chain	end	pick longest non-overlapping chain	<code>pair[0] > lastEnd</code> → pick	int (chain length)
56	Merge Intervals	start	extend or start fresh	<code>last.end < current.start</code> → new	int[]
57	Insert Interval	(no sort)	3 phases: before / overlap / after	<code>current.end < new.start</code> or <code>current.start > new.end</code>	int[]
25 3	Meeting Rooms II	start	reuse room if meeting already ended	<code>pq.peek() <= current.start</code> → reuse	int (rooms)
55	Jump Game	—	track max reachable index	<code>i > maxReach</code> → stuck	boolean
45	Jump Game II	—	BFS layers: extend window to farthest	<code>i == currentEnd</code> → jump	int (jumps)
76 3	Partition Labels	—	track last occurrence of each char	<code>i == end</code> → close partition	List
40 6	Queue Reconstruction by Height	height desc, k asc	insert each person at their k index	<code>result.add(p[1], p)</code>	int[]
13 4	Gas Station	Can complete circular route starting at some gas station	Greedy: track total gas surplus; if tank goes negative reset start; valid if total >= 0	Circular reset: if tank < 0, start = i+1, reset tank	O(n)
25 2	Meeting Rooms	Can person attend all meetings (no overlap)?	Sort by start time; check if intervals[i][0] < intervals[i-1][1]	Simple overlap check: sort by start, compare adjacent	O(n log n)

Two Interval Templates

MERGE DIRECTION RULE (for merge-style problems like #56, where both keys work):
Sort by START → traverse FORWARD (i = 0 → n-1), extend the END of last kept
Sort by END → traverse BACKWARD (i = n-1 → 0), extend the START of last kept
These two are exact mirror images. See #56 for both written out side by side.
(NOTE: this mirror only applies to merging. Template 1 below sorts by end but still sweeps FORWARD – there the goal is counting, not merging.)

TEMPLATE 1 – Sort by END time, sweep forward, track end of last kept interval
Use when: maximizing count of non-overlapping intervals
Greedy insight: earliest-finishing interval leaves most room for future ones

TEMPLATE 2 – Sort by START time, sweep forward, merge when overlapping
Use when: merging/counting overlapping intervals
Greedy insight: processing in start order → each interval only needs to compare with the last merged result

```

// MENTAL MODEL: sort to expose a local greedy choice, then sweep once making the locally-best pick.
// WHEN: intervals + "max non-overlapping / min arrows / merge / min rooms", or jump/partition sweeps.
// TEMPLATE 1: Sort by end, compare start
Arrays.sort(intervals, (a, b) -> a[1] - b[1]);    // sort by END time
int lastEnd = Integer.MIN_VALUE;
for (int[] interval : intervals) {
    if (interval[0] >= lastEnd) {                // start >= lastEnd → no overlap → keep
        lastEnd = interval[1];
        // count++ or add to result
    } else {
        // overlap → skip (or count as removed)
    }
}

// TEMPLATE 2: Sort by start, compare end
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);    // sort by START time
List<int[]> result = new ArrayList<>();
for (int[] interval : intervals) {
    if (result.isEmpty() || result.get(result.size()-1)[1] < interval[0]) {
        result.add(interval);                  // no overlap → new interval
    } else {
        result.get(result.size()-1)[1] =        // overlap → extend end
            Math.max(result.get(result.size()-1)[1], interval[1]);
    }
}

// TEMPLATE 3: Sort by start + min-heap of end times (multi-resource)
// USE WHEN counting simultaneous/overlapping resources – "minimum rooms", "max concurrent".
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap of active end times
for (int[] interval : intervals) {
    if (!pq.isEmpty() && pq.peek() <= interval[0])
        pq.poll();                             // reuse: earliest-ending resource is free
    pq.offer(interval[1]);                       // occupy a resource until interval[1]
}
// pq.size() = resources needed

```

Part 1 — Sort by End Time

#435 Non-overlapping Intervals

Description: Remove the minimum number of intervals to make the rest non-overlapping.

Intuition: Always keep the interval that finishes earliest — it leaves the most room for the rest, so greedily picking by end maximizes how many you keep.

Key: equivalent to keeping the maximum number of non-overlapping intervals. Sort by end, greedily keep each interval whose start $\geq$ last kept end.

```

class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]);    // sort by END
        int kept = 0, lastEnd = Integer.MIN_VALUE;
        for (int[] interval : intervals) {
            if (interval[0] >= lastEnd) {                // no overlap → keep
                lastEnd = interval[1];
                kept++;
            }
            // else: overlap → skip (counted as removed)
        }
        return intervals.length - kept;
    }
}

```

Time $O(n \log n)$ | **Space** $O(1)$

#452 Minimum Number of Arrows to Burst Balloons

Description: Balloons span `[start, end]`. An arrow shot at `x` pops all balloons with `start ≤ x ≤ end`. Minimum arrows to pop all balloons.

Intuition: Shoot the arrow at the earliest end point to pop every balloon overlapping it; only start a new arrow when a balloon begins past the current arrow.

Key: same structure as #435 — sort by end, count groups. One arrow suffices per group. New group starts when `start > current arrow position (= last group's end)`.

```
class Solution {
    public int findMinArrowShots(int[][] points) {
        Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1])); // sort by END (use compare to avoid overflow)
        int arrows = 1, arrowEnd = points[0][1];
        for (int[] point : points) {
            if (point[0] > arrowEnd) { // new group: no overlap with current arrow
                arrows++;
                arrowEnd = point[1];
            }
            // else: same arrow covers this balloon (start <= arrowEnd)
        }
        return arrows;
    }
}
```

Time $O(n \log n)$ | **Space** $O(1)$

#646 Maximum Length of Pair Chain

Description: Given pairs `[a, b]`, form a chain where `b < c` for each consecutive pair `[a, b] → [c, d]`. Maximum chain length.

Intuition: Picking the pair that ends earliest each time leaves the most slack for later links, maximizing the chain.

Key: identical to #435 / #452 — sort by end, greedily pick non-overlapping pairs.

```
class Solution {
    public int findLongestChain(int[][] pairs) {
        Arrays.sort(pairs, (a, b) -> a[1] - b[1]); // sort by END
        int count = 0, lastEnd = Integer.MIN_VALUE;
        for (int[] pair : pairs) {
            if (pair[0] > lastEnd) { // strict: pair[0] > lastEnd
                count++;
                lastEnd = pair[1];
            }
        }
        return count;
    }
}
```

Time $O(n \log n)$ | **Space** $O(1)$

435 vs 452 vs 646 — The One Difference

	#435 Non-overlapping	#452 Min Arrows	#646 Pair Chain
Sort by:	end	end	end
Keep condition:	<code>start >= lastEnd</code>	<code>start > arrowEnd</code>	<code>start > lastEnd</code>
Boundary:	<code>>=</code> (inclusive)	<code>></code> (exclusive)	<code>></code> (exclusive)
Return:	<code>intervals.length - kept arrows (groups)</code>	<code>count (groups)</code>	

Why 435 uses `>=` but 452/646 use `>`:

#435: touching intervals `[1,2], [2,3]` are non-overlapping (`gap = 0` is OK)

#452: arrow at `x=2` pops BOTH `[1,2]` and `[2,3]` (touching = same arrow)

#646: pair chain requires `b < c` strictly (touching `b==c` not allowed)

Part 2 — Sort by Start Time

#56 Merge Intervals

Description: Merge all overlapping intervals. Return the minimum set of non-overlapping intervals.

Intuition: sort to make overlaps adjacent, then sweep once — each interval only ever compares against the most recently kept one.

The direction rule — sort key dictates traversal direction:

- **Sort by START** → **traverse forward** ($i = 0 \rightarrow n-1$). The newest interval can only extend the last kept interval's **END**, so you compare `interval.start` against `last.end` and update `last[1]` .
- **Sort by END** → **traverse backward** ($i = n-1 \rightarrow 0$). The newest interval can only extend the last kept interval's **START**, so you compare `interval.end` against `last.start` and update `last[0]` .

Both are symmetric and equally valid. Pick the one whose sort key you already have.

Method A — Sort by START, traverse forward

Compare current `start` to last kept `end` ; on overlap, extend the **end**.

```
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // sort by START
        List<int[]> result = new ArrayList<>();
        for (int[] interval : intervals) { // ← traverse FORWARD
            if (result.isEmpty()) {
                result.add(interval);
                continue;
            }
            int[] last = result.get(result.size() - 1);
            if (last[1] < interval[0]) { // no overlap (last.end < current.start)
                result.add(interval);
            } else {
                last[1] = Math.max(last[1], interval[1]); // overlap → extend END
            }
        }
        return result.toArray(new int[0][]);
    }
}
```

Method B — Sort by END, traverse backward

Mirror image: compare current `end` to last kept `start` ; on overlap, extend the **start**.

```
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // sort by END
        List<int[]> result = new ArrayList<>();
        for (int i = intervals.length - 1; i >= 0; i--) { // ← traverse BACKWARD
            if (result.isEmpty()) {
                result.add(intervals[i]);
                continue;
            }
            int[] last = result.get(result.size() - 1);
            int[] current = intervals[i];
            if (last[0] > current[1]) { // no overlap (last.start > current.end)
                result.add(current);
            } else {
                last[0] = Math.min(last[0], current[0]); // overlap → extend START
            }
        }
        return result.toArray(new int[0][]);
    }
}
```

Time $O(n \log n)$ | **Space** $O(n)$

#57 Insert Interval

Description: Insert a new interval into a sorted list of non-overlapping intervals; merge if necessary. The input is already sorted — no sort needed.

Intuition: Walk the sorted list in three phases — copy everything that ends before the new interval, absorb everything that overlaps it, then copy the rest.

Key: three phases: (1) add all intervals that end before new starts, (2) merge all overlapping, (3) add remaining.

```
class Solution {
    public int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> list = new ArrayList<>();
        for(int[] interval : intervals) {
            if(interval[1] < newInterval[0]) {
                list.add(interval);
            } else if (interval[0] > newInterval[1]) {
                list.add(newInterval);
                newInterval = interval;
            } else {
                newInterval[0] = Math.min(newInterval[0], interval[0]);
                newInterval[1] = Math.max(newInterval[1], interval[1]);
            }
        }
        list.add(newInterval);
        return list.toArray(new int[list.size()][2]);
    }
}
```

Time $O(n)$ | **Space** $O(n)$

Part 3 — Sort by Start + Min-Heap (Multi-Resource)

#253 Meeting Rooms II

Description: Minimum number of meeting rooms required to host all meetings.

Intuition: USE WHEN counting simultaneous/overlapping resources — "minimum rooms", "max concurrent". The heap size at any moment is exactly how many meetings are running at once.

Template: sort by start. Min-heap tracks end times of active meetings. When a new meeting starts after the earliest-ending meeting finishes, reuse that room.

```
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // sort by START
        PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap of end times
        for (int[] interval : intervals) {
            if (!pq.isEmpty() && pq.peek() <= interval[0])
                pq.poll(); // ← reuse: earliest room is free
            pq.offer(interval[1]); // ← occupy room until this end
        }
        return pq.size(); // ← rooms in use = heap size
    }
}
```

Time $O(n \log n)$ | **Space** $O(n)$

Sort-by-End vs Sort-by-Start+Heap

	Sort by End	Sort by Start + Heap
Goal:	count non-overlapping	count simultaneous active
Tracks:	lastEnd (scalar)	all active end times (heap)
Question answered:	max intervals I can keep	min rooms (resources) needed
Examples:	#435, #452, #646	#253 Meeting Rooms

Part 4 — Non-interval Greedy

#55 Jump Game

Description: Each element is the max jump from that position. Can you reach the last index?

Intuition: Sweep left to right tracking the farthest index reachable; if you ever stand past that frontier you're stuck.

Key: track the farthest index reachable so far. If current index exceeds it, you're stuck.

```
class Solution {
    public boolean canJump(int[] nums) {
        int maxReach = 0;
        for (int i = 0; i < nums.length; i++) {
            if (i > maxReach) return false;           // ← can't reach index i
            maxReach = Math.max(maxReach, i + nums[i]);
        }
        return true;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#45 Jump Game II

Description: Minimum jumps to reach the last index.

Intuition: Treat each jump as a BFS layer — extend through the whole current reach before incrementing the jump count.

Key: BFS layer-by-layer. `currentEnd` = farthest we can reach with current jumps. `farthest` = farthest reachable with one more jump from anywhere in current layer. When we reach `currentEnd`, we must jump.

```
class Solution {
    public int jump(int[] nums) {
        int jumps = 0, currentEnd = 0, farthest = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);
            if (i == currentEnd) {                    // ← exhausted current layer
                jumps++;
                currentEnd = farthest;                 // ← extend to next layer
            }
        }
        return jumps;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#763 Partition Labels

Description: Partition string into as many parts as possible so each letter appears in at most one part. Return the size of each part.

Intuition: A partition can't close until you pass the last occurrence of every letter seen inside it, so extend `end` to that frontier and cut when you reach it.

Key: precompute the last occurrence of each character. Extend the current partition's end to `max(end, last[c])` for each character. Close partition when `i == end`.

```
class Solution {
    public List<Integer> partitionLabels(String s) {
        int[] last = new int[26];
        for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i; // ← last occurrence
        List<Integer> result = new ArrayList<>();
        int start = 0, end = 0;
        for (int i = 0; i < s.length(); i++) {
            end = Math.max(end, last[s.charAt(i) - 'a']); // ← must include last of this char
            if (i == end) { // ← reached end of partition
                result.add(end - start + 1);
                start = i + 1;
            }
        }
        return result;
    }
}
```

Time O(n) | Space O(1)

#406 Queue Reconstruction by Height

Description: People given as `[h, k]` (height `h`, `k` taller/equal people in front). Reconstruct the queue.

Intuition: Place tallest first so that inserting a person at index `k` is always correct — only taller/equal people are already placed, and shorter insertions later don't disturb their counts.

Key: sort descending by height (ties: ascending by `k`). Insert each person at index `k`. Since taller people are placed first, inserting at position `k` is always valid — there are already exactly `k` people $\geq$ this height already placed.

```
class Solution {
    public int[][] reconstructQueue(int[][] people) {
        Arrays.sort(people, (a, b) -> a[0] != b[0] ? b[0] - a[0] : a[1] - b[1]); // ← tall first, ties by k
        List<int[]> result = new ArrayList<>();
        for (int[] p : people)
            result.add(p[1], p); // ← insert at index k
        return result.toArray(new int[0][]);
    }
}
```

Time O(n²) | Space O(n)

Greedy Pattern Identifier

Signal	Template
"minimum removal to make non-overlapping"	Sort by end, keep non-overlapping (Template 1)
"minimum resources (arrows, groups)"	Sort by end, count groups (Template 1)
"merge overlapping intervals"	Sort by start, extend or add (Template 2)
"minimum rooms/machines"	Sort by start + min-heap (Template 3)
"can you reach the end?"	Track max reachable index
"minimum steps to reach the end?"	BFS layers via <code>currentEnd</code>
"partition string by last occurrence"	Track <code>last[]</code> array + extend <code>end</code>
"reconstruct order by two constraints"	Sort by one constraint, insert by the other
"can you complete the circuit / circular route?"	Track cumulative tank; reset start when tank < 0
"can person attend all meetings (no overlap)?"	Sort by start, compare adjacent intervals

Part 5 — Overlap & Circular Patterns

#252 Meeting Rooms

Description: Given an array of meeting intervals, determine if a person could attend all meetings (no two meetings overlap).

Intuition: After sorting by start, any overlap must be between adjacent intervals, so one linear scan comparing neighbors is enough.

Algorithm: Sort by start time. If any meeting starts before the previous one ends, there is an overlap — return false.

```
class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);    // ← sort by start time
        for (int i = 1; i < intervals.length; i++)
            if (intervals[i][0] < intervals[i-1][1]) return false;    // ← VARIATION: start < previous end = overlap
        return true;
    }
}
```

Time $O(n \log n)$ | **Space** $O(1)$

#134 Gas Station

Description: There are n gas stations on a circular route. `gas[i]` is the gas at station i ; `cost[i]` is the gas needed to travel from i to $i+1$. Return the starting station index if you can complete the circuit, or -1 if not.

Intuition: If the tank ever drops below 0, no station between the old start and here can work, so jump the start past the failure point; a valid start exists iff total gas $\geq$ total cost.

Algorithm: Greedy. Track cumulative tank. When tank goes negative, reset start to `i+1` and reset tank. If total gas $\geq$ total cost, a valid start exists (the last reset start).

```
class Solution {
    public int canCompleteCircuit(int[] gas, int[] cost) {
        int total = 0, tank = 0, start = 0;
        for (int i = 0; i < gas.length; i++) {
            int diff = gas[i] - cost[i];
            total += diff;
            tank += diff;
            if (tank < 0) {
                start = i + 1;    // ← VARIATION: reset start when tank goes negative
                tank = 0;
            }
        }
        return total >= 0 ? start : -1;    // ← VARIATION: if total >= 0, solution exists
    }
}
```

Time $O(n)$ | **Space** $O(1)$